Recurrent Neural Networks

Mathematics, Algorithms, and Applications

Morten Hjorth-Jensen

FYS5429/9429 March 5

Why Sequential Models?

Many physical and computational problems involve sequences:

- time series analysis
- natural language processing
- dynamical systems
- signal processing
- control problems

Examples of applications in the physical sciences:

- molecular dynamics trajectories
- quantum time evolution
- climate modelling
- detector signals

Standard feed-forward networks assume independent samples.

Why sequential models?

Many datasets are *ordered* and exhibit temporal/causal structure:

- time series in physics and engineering (signals, sensors, trajectories)
- language (sentences), speech, audio
- dynamical systems and control

A feed-forward network typically assumes:

- fixed-size input vectors,
- exchangeability (ordering does not matter),
- no internal memory.

RNNs introduce a **state** that evolves with the sequence.

Sequential data notation

We represent an input sequence as

$$x_{1:T} \equiv (x_1, x_2, \ldots, x_T), \qquad x_t \in \mathbb{R}^d.$$

Typical prediction settings:

- **many-to-one:** predict a label from the whole sequence
- **one-to-many:** generate a sequence from one input
- **many-to-many:** sequence tagging or forecasting

Core challenge: capture dependencies across time steps t.

Sources and pointers (for students)

Standard references:

- I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning* (RNN chapter)
- C. Bishop, *Pattern Recognition and Machine Learning* (sequence models background)
- S. Hochreiter & J. Schmidhuber (1997): LSTM
- K. Cho et al. (2014): GRU

What is an RNN?

An RNN maintains a **hidden state** $h_t \in \mathbb{R}^m$ that summarizes the past:

$$h_t = f_\theta(x_t, h_{t-1}).$$

Key idea: parameters θ are *shared* across time steps.

- compact parameterization for long sequences
- natural inductive bias for temporal structure

Why RNNs?

RNNs are designed to learn:

- **short-range dependencies** (local patterns)
- **long-range dependencies** (delayed effects, memory)

In varios applciations this resembles:

- state-space models
- discrete-time dynamical systems
- Markovian updates with learnable transition maps

Limitations of vanilla RNNs

Training a basic RNN can be difficult due to:

- **vanishing gradients** (long-range information fades)
- **exploding gradients** (instability)
- optimization challenges in deep unrolled computation graphs

Modern gated architectures (LSTM/GRU) mitigate these issues.

Sequential Data Structure

Instead of a single input vector:

$$x \in \mathbb{R}^d$$

we have a sequence

$$x_1, x_2, \ldots, x_T$$

Each element:

$$x_t \in \mathbb{R}^d$$

represents the system at time step t.

Core Idea

Recurrent neural networks introduce a hidden state

$$h_t$$

that summarizes the past.

$$h_t = f(x_t, h_{t-1})$$

The hidden state acts as a memory.

RNN Update Equations

Standard formulation:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

Output:

$$y_t = \phi(W_{hy}h_t + b_y)$$

Parameters:

$$\theta = \{W_{xh}, W_{hh}, W_{hy}, b_h, b_y\}$$

Vanilla RNN architecture (vector form)

Let $x_t \in \mathbb{R}^d$, hidden state $h_t \in \mathbb{R}^m$, output pre-activation $o_t \in \mathbb{R}^k$. Define

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h, \qquad h_t = \sigma(a_t),$$

$$o_t = W_{hy}h_t + b_y, \qquad y_t = \phi(o_t).$$

Parameters:

$$W_{xh} \in \mathbb{R}^{m\times d},\ W_{hh} \in \mathbb{R}^{m\times m},\ W_{hy} \in \mathbb{R}^{k\times m}.$$

Unrolling through time

The recurrence couples time steps:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h).$$

Unrolling yields a depth-T computation graph with weight sharing:

- forward pass computes $h_1, \ldots, h_T$ sequentially
- backward pass propagates sensitivities from T back to 1

Initialization: $h_0 = 0$ (or learn h_0).

Loss functions for sequences

Common objective:

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t(y_t, \hat{y}_t).$$

Examples:

- regression: $\ell_t = \frac{1}{2}\|y_t - \hat{y}_t\|^2$
- classification: $\ell_t = -\sum_i \hat{y}_{t,i} \log y_{t,i}$

We now derive gradients for training via **backpropagation through time** (BPTT).

Forward Propagation

For each time step:

$$h_t = \sigma(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$y_t = \phi(W_{hy}h_t + b_y)$$

Initial state:

$$h_0 = 0$$

or learned parameters.

Unrolling the Network

RNNs can be viewed as deep networks over time.

$$x_1 \rightarrow h_1 \rightarrow y_1$$

$$x_2 \rightarrow h_2 \rightarrow y_2$$

$$x_3 \rightarrow h_3 \rightarrow y_3$$

Weights are shared across time.

Sequence Loss

Define total loss:

$$L = \sum_{t=1}^{T} \ell(y_t, \hat{y}_t)$$

Examples:
Mean squared error:

$$\ell = ||y_t - \hat{y}_t||^2$$

Cross entropy:

$$\ell = -\sum_i \hat{y}_i \log y_i$$

Training the Network

Training requires computing

$$\frac{\partial L}{\partial \theta}$$

Since hidden states depend on earlier states, gradients must propagate through time.
This algorithm is called
Backpropagation Through Time (BPTT).

Hidden State Gradient

Define

$$\delta_t = \frac{\partial L}{\partial h_t}$$

Using the chain rule:

$$\delta_t = \frac{\partial \ell_t}{\partial h_t} + W_{hh}^T \delta_{t+1}$$

Weight Gradients

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \delta_t h_{t-1}^T$$

$$\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} \delta_t x_t^T$$

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} \frac{\partial \ell_t}{\partial y_t} h_t^T$$

Vanishing Gradients

Backpropagation involves repeated multiplication:

$$(W_{hh})^k$$

If eigenvalues satisfy

$$|\lambda| < 1$$

gradients decay exponentially.

Exploding Gradients

If

$$|\lambda| > 1$$

gradients grow exponentially.
Solutions:

- gradient clipping
- improved architectures
- careful initialization

Setup and Notation (Vectorized)

We consider a (single-sequence) vanilla RNN with hidden size m, input size d, output size k.
Forward equations:

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h, \qquad h_t = \sigma(a_t)$$

$$o_t = W_{hy}h_t + b_y, \qquad y_t = \phi(o_t)$$

Dimensions:

$$W_{xh} \in \mathbb{R}^{m\times d}, \quad W_{hh} \in \mathbb{R}^{m\times m}, \quad b_h \in \mathbb{R}^m$$

$$W_{hy} \in \mathbb{R}^{k\times m}, \quad b_y \in \mathbb{R}^k$$

Total loss (sequence-to-sequence):

$$L = \sum_{t=1}^{T} \ell_t(y_t, \hat{y}_t).$$

Matrix Calculus Rules Used

We use the following standard identities:

- For $z = Ax$, $\frac{\partial L}{\partial x} = A^T \frac{\partial L}{\partial z}$.
- For $z = Ax + b$, $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial z} x^T$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$.
- For elementwise nonlinearity $h = \sigma(a)$:

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial h} \odot \sigma'(a),$$

  where $\odot$ is Hadamard product.

We will compute gradients by backpropagating through the unrolled computational graph.

Local Output Gradients

Define output-layer "error signals":

$$\delta_t^o \equiv \frac{\partial L}{\partial o_t} \in \mathbb{R}^k.$$

By chain rule:

$$\delta_t^o = \left(\frac{\partial \ell_t}{\partial y_t}\right) \odot \phi'(o_t).$$

Then (matrix calculus):

$$\frac{\partial L}{\partial W_{hy}} = \sum_{t=1}^{T} \delta_t^o \, h_t^T, \qquad \frac{\partial L}{\partial b_y} = \sum_{t=1}^{T} \delta_t^o.$$

Hidden contribution from output at time t:

$$\left.\frac{\partial L}{\partial h_t}\right|_{\text{out}} = W_{hy}^T \delta_t^o.$$

Key Quantity: Hidden-State Sensitivity

We want the total sensitivity

$$\delta_t^h \equiv \frac{\partial L}{\partial h_t} \in \mathbb{R}^m.$$

Because h_t influences:

- the loss at time t via o_t,
- all future hidden states $h_{t+1}, h_{t+2}, \ldots$ through recurrence.

Thus:

$$\delta_t^h = \left.\frac{\partial L}{\partial h_t}\right|_{\text{out}} + \left.\frac{\partial L}{\partial h_t}\right|_{\text{future}}.$$

We now compute the future term via the recurrence Jacobian.

From δ_t^h to δ_t^a

Define the pre-activation error:

$$\delta_t^a \equiv \frac{\partial L}{\partial a_t} \in \mathbb{R}^m.$$

Since $h_t = \sigma(a_t)$ elementwise:

$$\delta_t^a = \delta_t^h \odot \sigma'(a_t).$$

This δ_t^a is the central object used to compute parameter gradients in the recurrent core.

Recurrence Jacobian and Backward Recursion

We have

$$a_{t+1} = W_{xh}x_{t+1} + W_{hh}h_t + b_h.$$

Hence the Jacobian of a_{t+1} w.r.t. h_t is:

$$\frac{\partial a_{t+1}}{\partial h_t} = W_{hh}.$$

Using matrix calculus:

$$\left.\frac{\partial L}{\partial h_t}\right|_{\text{future}} = \left(\frac{\partial a_{t+1}}{\partial h_t}\right)^T \frac{\partial L}{\partial a_{t+1}} = W_{hh}^T \delta_{t+1}^a.$$

Therefore the BPTT recursion is:

$$\boxed{\delta_t^h = W_{hy}^T \delta_t^o + W_{hh}^T \delta_{t+1}^a}, \qquad \boxed{\delta_t^a = \delta_t^h \odot \sigma'(a_t)}$$

with terminal condition $\delta_{T+1}^a = 0$.

Core Parameter Gradients via Matrix Calculus

Recall:

$$a_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h.$$

Using the identity $\partial L/\partial A = (\partial L/\partial z)\, x^T$ for $z = Ax$:

$$\boxed{\frac{\partial L}{\partial W_{xh}} = \sum_{t=1}^{T} \delta_t^a\, x_t^T}$$

$$\boxed{\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \delta_t^a\, h_{t-1}^T}$$

$$\boxed{\frac{\partial L}{\partial b_h} = \sum_{t=1}^{T} \delta_t^a}$$

These are the standard BPTT gradients for a vanilla RNN.

Explicit "Unrolled" Form (Useful for Analysis)

Iterating the recursion:

$$\delta_t^h = W_{hy}^T \delta_t^o + W_{hh}^T (\delta_{t+1}^h \odot \sigma'(a_{t+1})).$$

Expanding gives:

$$\delta_t^h = W_{hy}^T \delta_t^o + W_{hh}^T D_{t+1} W_{hy}^T \delta_{t+1}^o + W_{hh}^T D_{t+1} W_{hh}^T D_{t+2} W_{hy}^T \delta_{t+2}^o + \cdots$$

where $D_t = \mathrm{diag}(\sigma'(a_t))$.
This makes the role of repeated Jacobian products explicit.

Vanishing/Exploding Gradients from Jacobian Products

The "future" influence involves products like:

$$(W_{hh}^T D_{t+1})(W_{hh}^T D_{t+2}) \cdots (W_{hh}^T D_{t+k}).$$

If the operator norm satisfies (roughly)

$$\|W_{hh}\| \cdot \max_t \|D_t\| < 1,$$

then gradients shrink exponentially (vanishing gradients).
If

$$\|W_{hh}\| \cdot \max_t \|D_t\| > 1,$$

then gradients can grow exponentially (exploding gradients).
Mitigation strategies:

- gradient clipping
- orthogonal/unitary W_{hh}
- gating (LSTM/GRU)

BPTT Algorithm (Practical Implementation)

Forward pass (store a_t, h_t, o_t, y_t for $t = 1, \ldots, T$).
Backward pass:

1. Initialize $\delta^a_{T+1} = 0$.
2. For $t = T, \ldots, 1$:
   - compute $\delta^o_t = \frac{\partial \ell_t}{\partial y_t} \odot \phi'(o_t)$
   - compute $\delta^h_t = W^T_{hy}\delta^o_t + W^T_{hh}\delta^a_{t+1}$
   - compute $\delta^a_t = \delta^h_t \odot \sigma'(a_t)$
   - accumulate gradients:

$$\nabla W_{hy} \mathrel{+}= \delta^o_t h^T_t, \quad \nabla b_y \mathrel{+}= \delta^o_t$$

$$\nabla W_{xh} \mathrel{+}= \delta^a_t x^T_t, \quad \nabla W_{hh} \mathrel{+}= \delta^a_t h^T_{t-1}, \quad \nabla b_h \mathrel{+}= \delta^a_t$$

Truncated BPTT

For very long sequences, backpropagating through all T steps is costly and unstable.

Truncated BPTT:

- Split sequence into blocks of length K.
- Backpropagate only K steps at a time.

Effect:

- reduces cost from $O(T)$ memory to $O(K)$
- limits long-range credit assignment

Motivation for LSTM

Simple RNNs struggle with long-range dependencies.
LSTMs introduce gates that control information flow.

LSTM Equations

Forget gate:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

Input gate:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

Candidate state:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1})$$

Cell State Update

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Output gate:

$$o_t = \sigma(W_o x_t + U_o h_{t-1})$$

Hidden state:

$$h_t = o_t \odot \tanh(c_t)$$

GRU Architecture

Simplified alternative to LSTM.
Update gate:

$$z_t = \sigma(W_z x_t + U_z h_{t-1})$$

Reset gate:

$$r_t = \sigma(W_r x_t + U_r h_{t-1})$$

GRU Update

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}))$$

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t$$

Computational Complexity

Cost per time step:

$$O(n_h^2)$$

where n_h is the hidden dimension.
Total cost:

$$O(Tn_h^2)$$

RNN applications

Examples:

- quantum state reconstruction
- time-series modelling
- turbulence prediction
- particle detector signals
- dynamical system forecasting

Example: Dynamical Systems

Given data

$$x_{t+1} = F(x_t)$$

RNN learns

$$x_{t+1} \approx f_\theta(x_t, h_t)$$

This provides a learned surrogate model.

Summary

- RNNs model sequential data using hidden states
- training uses backpropagation through time
- gradients can vanish or explode
- LSTM and GRU mitigate these problems
- widely used in physics and machine learning